Sprint 2 Notes: Document Boilerplate and Head Metadata

Sprint 2 Goal

The goal of Sprint 2 is to learn how to write a complete HTML document.

In Sprint 1, you learned how to write individual HTML elements, such as headings, paragraphs, images, attributes, `id`, `class`, and void elements.

In Sprint 2, you learn where those elements belong inside a real HTML page.

By the end of this sprint, you should be able to:

- Write a full HTML boilerplate from memory.
- Explain what `DOCTYPE` does.
- Use the `html`, `head`, and `body` elements correctly.
- Add important metadata inside the `head`.
- Add visible content inside the `body`.
- Add `charset` and viewport meta tags.
- Add a page title.
- Add a meta description.
- Link an external CSS file.
- Link a favicon.
- Recognize preconnect links for external resources.
- Recognize Open Graph metadata.
- Recognize script references without learning JavaScript yet.
- Understand separation of concerns.

---

1. Why Sprint 2 Matters

A few HTML elements by themselves are not a full web page.

Example from Sprint 1:

```
<h1>My Page</h1>
<p>This is a paragraph.</p>
<img src="cat.jpg" alt="A cat sleeping.">
```

This is valid HTML content, but it is not a complete HTML document.

A complete HTML document needs a larger structure around the visible content.

That structure tells the browser:

- What type of document this is.
- What language the page uses.
- What metadata belongs to the page.
- What title appears in the browser tab.
- What CSS file should style the page.
- What content should be visible to the user.

Sprint 2 teaches that structure.

---

2. Full HTML Boilerplate

A boilerplate is a reusable starting structure.

When you create a new HTML file, you usually begin with this pattern:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Document Title Goes Here</title>
    <link rel="stylesheet" href="./styles.css">
  </head>

  <body>
    <h1>I am a main title</h1>
    <p>Example paragraph text.</p>
  </body>
</html>
```

This is the main structure you need to memorize.

Simple mental model:

```
<!DOCTYPE html>  = tells the browser to use modern HTML rules
<html>           = wraps the whole page
<head>           = information about the page
<body>           = visible page content
```

3. `<!DOCTYPE html>`

The first line of a modern HTML document should be:

```
<!DOCTYPE html>
```

This tells the browser:

```
This document uses modern HTML.
```

It helps the browser render the page using modern standards.

Correct:

```
<!DOCTYPE html>
<html lang="en">
```

Incorrect:

```
<html lang="en">
<!DOCTYPE html>
```

Why incorrect?

Because `DOCTYPE` must come first.

Important rule:

```
Always put <!DOCTYPE html> at the very top of the HTML file.
```

---

4. The `html` Element

The `html` element wraps the whole document.

Example:

```
<html lang="en">
  ...
</html>
```

Everything in the page goes inside the `html` element, except the `DOCTYPE`.

The basic structure is:

```
<!DOCTYPE html>
<html lang="en">
  <head>
  </head>

  <body>
  </body>
</html>
```

The `html` element usually has a `lang` attribute.

---

5. The `lang` Attribute

The `lang` attribute tells browsers, search engines, and assistive technologies the main language of the page.

Example:

```
<html lang="en">
```

This means:

```
The main language of this page is English.
```

Other examples:

```
<html lang="es">
```

Spanish.

```
<html lang="si">
```

Sinhala.

```
<html lang="ta">
```

Tamil.

Use the language code that matches the main content of the page.

Why `lang` matters:

- Screen readers can pronounce text more correctly.
- Search engines can understand the page language.
- Browsers can apply language-specific behavior.
- Translation tools can detect the language more reliably.

Important rule:

```
Every full HTML document should have a lang attribute on the html element.
```

---

6. The `head` Element

The `head` element contains information about the page.

Example:

```
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
```

Most things inside `head` are not visible on the web page.

The `head` is for:

- Metadata.
- Character encoding.
- Viewport settings.
- Browser tab title.

- Page description.
- CSS links.
- Favicon links.
- Open Graph social preview tags.
- Resource hints such as preconnect.
- Script references.

The `head` is not for normal visible page content.

Incorrect:

```
<head>
  <h1>Welcome to My Page</h1>
</head>
```

Why incorrect?

Because `h1` is visible content. It belongs in the `body`.

Correct:

```
<body>
  <h1>Welcome to My Page</h1>
</body>
```

Simple rule:

```
head = information about the page
body = content shown on the page
```

---

7. The `body` Element

The `body` element contains the visible content of the page.

Example:

```
<body>
  <h1>Welcome</h1>
```

```
  <p>This text appears on the page.</p>
</body>
```

Put these inside the `body`:

- Headings.
- Paragraphs.
- Images.
- Links.
- Forms.
- Tables.
- Sections.
- Articles.
- Navigation.
- Footer content.
- Any content users should see or interact with.

Incorrect:

```
<head>
  <p>This paragraph should be visible.</p>
</head>
```

Correct:

```
<body>
  <p>This paragraph should be visible.</p>
</body>
```

Important rule:

```
If the user should see it, it usually belongs in body.
```

---

8. Metadata

Metadata means information about the page.

Metadata is usually placed inside the `head`.

Examples of metadata:

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="A short page description.">
```

Metadata can tell the browser:

- How to read text characters.
- How to display the page on different devices.
- What the page is about.
- How the page should appear when shared.

Metadata is important, but most metadata is not directly shown as visible content on the page.

---

9. `meta charset="UTF-8"`

The charset meta tag tells the browser what character encoding to use.

Use this:

```
<meta charset="UTF-8">
```

UTF-8 is a character encoding.

In simple words:

```
UTF-8 helps the browser display text correctly.
```

It supports many languages, characters, and symbols.

Examples:

```
English: Hello
Spanish: acción
Sinhala: ආයුබෝවන්
Tamil: வணக்கம்
Symbols: © € Ω
```

Without proper character encoding, some characters may display incorrectly.

Important rule:

```
Put <meta charset="UTF-8"> inside the head of every new HTML document.
```

Good placement:

```
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
```

---

10. Unicode and UTF-8

Unicode is a system that gives characters a standard identity.

UTF-8 is a common way to store and read Unicode characters.

Beginner version:

```
Unicode = the large character system
UTF-8 = the common encoding used to display those characters correctly
```

You do not need to master character encoding deeply in Sprint 2.

For now, remember this:

```
UTF-8 helps your page handle normal text, symbols, and many languages correctly.
```

---

11. Viewport Meta Tag

The viewport meta tag helps the page display properly on different screen sizes.

Use this:

```
<meta name="viewport" content="width=device-width, initial-scale=1.0">
```

Breakdown:

```
name="viewport"
```

This means the metadata is about the viewport.

The viewport is the visible area of the web page in the browser.

```
content="width=device-width, initial-scale=1.0"
```

This means:

```
Use the device's actual screen width.
Start with normal zoom.
```

Why this matters:

- Phones have smaller screens.
- Tablets have medium screens.
- Desktops have larger screens.
- The browser needs instructions for responsive layout.

Important rule:

```
Include the viewport meta tag in modern HTML pages.
```

Good placement:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
</head>
```

---

12. The `title` Element

The `title` element sets the text shown in the browser tab.

Example:

```
<title>Local Bakery</title>
```

The `title` belongs inside the `head`.

Example:

```
<head>
  <title>Local Bakery</title>
</head>
```

Do not confuse `title` with `h1`.

`title`:

```
<title>Local Bakery</title>
```

Appears in the browser tab.

`h1`:

```
<h1>Local Bakery</h1>
```

Appears visibly on the page.

A page usually has both:

```
<head>
  <title>Local Bakery</title>
</head>

<body>
  <h1>Local Bakery</h1>
</body>
```

Important rule:

```
Use title for the browser tab. Use h1 for the visible main page heading.
```

---

13. Meta Description

The meta description gives a short description of the page.

Example:

```
<meta name="description" content="A local bakery landing page with fresh bread,
cakes, and contact information.">
```

It belongs inside the `head`.

Example:

```
<head>
  <meta name="description"
content="A beginner HTML practice page about document structure.">
</head>
```

A meta description may be used as a search result snippet.

It should be:

- Short.
- Clear.
- Specific.
- Useful to a human reader.

Good:

```
<meta name="description" content="A local bakery landing page with fresh bread,
cakes, and contact information.">
```

Weak:

```
<meta name="description" content="This is my page and it is very nice and has
many things.">
```

Why weak?

Because it is vague.

Important rule:

```
Write meta descriptions for humans first. Keep them clear and specific.
```

---

14. The `link` Element

The `link` element connects the HTML document to an external resource.

Example:

```
<link rel="stylesheet" href="./styles.css">
```

The `link` element is a void element.

That means it does not have a closing tag.

Correct:

```
<link rel="stylesheet" href="./styles.css">
```

Incorrect:

```
<link rel="stylesheet" href="./styles.css"></link>
```

The `link` element usually goes inside the `head`.

---

15. `rel` and `href` on `link`

The `link` element commonly uses two important attributes:

```
rel="stylesheet"
href="./styles.css"
```

`rel` means relationship.

It tells the browser what kind of resource is being linked.

Example:

```
rel="stylesheet"
```

This means:

```
The linked file is a CSS stylesheet.
```

`href` gives the path or URL to the resource.

Example:

```
href="./styles.css"
```

This means:

```
Find a file named styles.css in the current folder.
```

Complete example:

```
<link rel="stylesheet" href="./styles.css">
```

Meaning:

```
Connect this HTML page to the CSS file named styles.css.
```

---

16. Linking an External Stylesheet

A stylesheet is a CSS file.

CSS controls the appearance of the page.

HTML controls structure.

Example:

```
<link rel="stylesheet" href="./styles.css">
```

This line should go inside the `head`.

Example:

```
<head>
  <link rel="stylesheet" href="./styles.css">
</head>
```

This keeps HTML and CSS separate.

Good idea:

```
HTML file = structure
CSS file = design
```

This is easier to maintain than putting everything in one file.

---

17. Favicon

A favicon is the small icon shown in the browser tab.

Example:

```
<link rel="icon" href="favicon.ico">
```

This belongs inside the `head`.

Example:

```
<head>
  <link rel="icon" href="./favicon.ico">
</head>
```

Breakdown:

```
rel="icon"
```

Means:

```
The linked file is the page icon.
```

```
href="./favicon.ico"
```

Means:

```
Use the file favicon.ico from the current folder.
```

Important rule:

```
Use link rel="icon" to connect a favicon.
```

---

18. Multiple `link` Elements

A page can have more than one `link` element.

Example:

```
<link rel="stylesheet" href="./styles.css">
<link rel="icon" href="./favicon.ico">
```

This is normal.

One `link` connects the stylesheet.

Another `link` connects the favicon.

More `link` elements may connect fonts or other resources.

---

19. Preconnect

Preconnect is a resource hint.

It tells the browser to prepare a connection to another website early.

Example:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

Beginner explanation:

```
The browser is being told: We will probably need files from this external place
soon, so prepare the connection early.
```

Preconnect is often used with external fonts.

You do not need to master performance optimization in Sprint 2.

For now, recognize this pattern:

```
<link rel="preconnect" href="https://example.com">
```

Important idea:

```
Preconnect helps prepare external resource loading earlier.
```

---

20. `crossorigin`

Sometimes preconnect uses the `crossorigin` attribute.

Example:

```
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
```

`crossorigin` is used when the browser needs to make a cross-origin request.

Beginner version:

```
crossorigin helps the browser handle certain external resources correctly.
```

You do not need to deeply understand cross-origin requests yet.

For Sprint 2, just recognize that it may appear with external resources such as fonts.

---

21. External Font Pattern

A common external font pattern looks like this:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?
family=Playwrite+CU:wght@100..400&display=swap" rel="stylesheet">
```

What this means:

- The first two lines prepare connections to Google font servers.
- The third line links the actual font stylesheet.

Important Sprint 2 point:

```
The font is linked in HTML, but font styling should be handled by CSS.
```

Do not put visual styling directly into the HTML just because you linked a font.

---

22. Open Graph Metadata

Open Graph metadata controls how a page can appear when shared on social platforms.

Basic Open Graph pattern:

```
<meta property="og:title" content="Site title">
<meta property="og:type" content="website">
```

```
<meta property="og:image" content="https://example.com/social-preview.png">
<meta property="og:url" content="https://example.com">
```

These tags belong inside the `head`.

The four core Open Graph properties in Sprint 2 are:

```
og:title
og:type
og:image
og:url
```

---

23. `og:title`

Example:

```
<meta property="og:title" content="Local Bakery">
```

This controls the title that may appear in a social preview.

Simple meaning:

```
When this page is shared, use Local Bakery as the preview title.
```

---

24. `og:type`

Example:

```
<meta property="og:type" content="website">
```

This tells the social platform what kind of thing the page is.

For a normal website page, use:

```
content="website"
```

25. `og:image`

Example:

```
<meta property="og:image" content="https://example.com/bakery-preview.png">
```

This gives the image that may appear in the social preview.

Important note:

```
Open Graph image URLs are commonly absolute URLs.
```

That means they include the full web address.

Example:

```
https://example.com/bakery-preview.png
```

---

26. `og:url`

Example:

```
<meta property="og:url" content="https://example.com">
```

This tells the social platform the main URL of the page.

Simple meaning:

```
This is the official URL for this page.
```

---

27. The `script` Element

The `script` element is used for JavaScript.

Example:

```
<script src="./script.js"></script>
```

In Sprint 2, you do not need to learn JavaScript.

You only need to recognize how an external script can be linked.

The `src` attribute points to the JavaScript file.

Example:

```
<script src="path-to-javascript-file.js"></script>
```

Unlike `link`, the `script` element usually has a closing tag.

Correct:

```
<script src="./script.js"></script>
```

Important idea:

```
HTML can reference JavaScript, but this sprint does not teach JavaScript
behavior.
```

---

28. Separation of Concerns

Separation of concerns means keeping different responsibilities separate.

Simple version:

```
HTML = structure and content
CSS = appearance and design
JavaScript = behavior and interactivity
```

Example of separated files:

```
index.html   = page structure
styles.css   = page design
script.js    = page behavior
```

Why separation is useful:

- Code is easier to read.
- Code is easier to maintain.
- Different files have clear jobs.
- You can change styling without rewriting HTML structure.
- You can change behavior without mixing it into page content.

Good pattern:

```
<link rel="stylesheet" href="./styles.css">
<script src="./script.js"></script>
```

This keeps CSS and JavaScript outside the HTML file.

Important Sprint 2 rule:

```
Do not put everything inside HTML when an external file is more maintainable.
```

---

29. Metadata vs Visible Content

A common beginner mistake is mixing metadata and visible content.

Metadata belongs in `head`.

Visible content belongs in `body`.

Metadata examples:

```
<meta charset="UTF-8">
<meta name="viewport" content="width=device-width, initial-scale=1.0">
<meta name="description" content="A short page description.">
<title>My Page</title>
<link rel="stylesheet" href="./styles.css">
```

Visible content examples:

```
<h1>My Page</h1>
<p>This is visible text.</p>
<img src="cat.jpg" alt="A cat sleeping.">
```

Incorrect:

```
<head>
  <h1>My Page</h1>
  <p>This is visible text.</p>
</head>
```

Correct:

```
<head>
  <title>My Page</title>
</head>

<body>
  <h1>My Page</h1>
  <p>This is visible text.</p>
</body>
```

---

30. Complete Beginner Boilerplate

This is the main Sprint 2 boilerplate:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>My First Full HTML Page</title>
    <link rel="stylesheet" href="./styles.css">
  </head>

  <body>
    <h1>My First Full HTML Page</h1>
    <p>I am learning how to build a complete HTML document.</p>
```

```
  </body>
</html>
```

You should be able to write this from memory.

---

31. Expanded Boilerplate with Description and Favicon

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Local Bakery</title>
    <meta name="description" content="A local bakery landing page with fresh
bread, cakes, and contact information.">

    <link rel="stylesheet" href="./styles.css">
    <link rel="icon" href="./favicon.ico">
  </head>

  <body>
    <h1>Local Bakery</h1>
    <p>Fresh bread, cakes, and pastries made every morning.</p>
  </body>
</html>
```

This version includes:

- `DOCTYPE`.
- `html lang`.
- `head`.
- `body`.
- UTF-8 charset.
- Viewport meta tag.
- Page title.
- Meta description.
- Stylesheet link.
- Favicon link.
- Visible heading.
- Visible paragraph.

32. Expanded Boilerplate with Open Graph

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Local Bakery</title>
    <meta name="description" content="A local bakery landing page with fresh
bread, cakes, and contact information.">

    <link rel="stylesheet" href="./styles.css">
    <link rel="icon" href="./favicon.ico">

    <meta property="og:title" content="Local Bakery">
    <meta property="og:type" content="website">
    <meta property="og:image" content="https://example.com/bakery-preview.png">
    <meta property="og:url" content="https://example.com">
  </head>

  <body>
    <h1>Local Bakery</h1>
    <p>Fresh bread, cakes, and pastries made every morning.</p>
  </body>
</html>
```

This is a strong Sprint 2 example.

---

33. Order of Main Document Parts

Use this order:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <!-- metadata and external resource links -->
  </head>

  <body>
    <!-- visible page content -->
```

```
  </body>
</html>
```

Correct order:

```
1. DOCTYPE
2. html opening tag
3. head
4. body
5. html closing tag
```

Incorrect order:

```
<html>
  <body>
  </body>

  <head>
  </head>
</html>
```

Why incorrect?

Because `head` should come before `body`.

---

34. Common Mistakes to Avoid

Mistake 1: Forgetting `DOCTYPE`

Incorrect:

```
<html lang="en">
```

Better:

```
<!DOCTYPE html>
<html lang="en">
```

Mistake 2: Missing `lang`

Weak:

```
<html>
```

Better:

```
<html lang="en">
```

Mistake 3: Putting metadata in `body`

Incorrect:

```
<body>
  <meta charset="UTF-8">
  <title>My Page</title>
</body>
```

Correct:

```
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
```

Mistake 4: Putting visible content in `head`

Incorrect:

```
<head>
  <h1>Hello</h1>
</head>
```

Correct:

```
<body>
  <h1>Hello</h1>
</body>
```

Mistake 5: Forgetting `UTF-8`

Weak:

```
<head>
  <title>My Page</title>
</head>
```

Better:

```
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
```

Mistake 6: Forgetting viewport

Weak:

```
<head>
  <meta charset="UTF-8">
  <title>My Page</title>
</head>
```

Better:

```
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>My Page</title>
</head>
```

Mistake 7: Writing vague meta descriptions

Weak:

```
<meta name="description" content="This is my nice page.">
```

Better:

```
<meta name="description" content="A beginner HTML page explaining document
boilerplate and metadata.">
```

Mistake 8: Closing a `link` element

Incorrect:

```
<link rel="stylesheet" href="./styles.css"></link>
```

Correct:

```
<link rel="stylesheet" href="./styles.css">
```

Mistake 9: Mixing CSS directly into HTML unnecessarily

Weak for maintainability:

```
<style>
  body {
    font-family: Arial, sans-serif;
  }
</style>
```

Better for separation of concerns:

```
<link rel="stylesheet" href="./styles.css">
```

Mistake 10: Thinking Sprint 2 requires JavaScript

Sprint 2 only asks you to recognize the `script` element.

You do not need to learn JavaScript yet.

---

35. Debugging Sprint 2 Broken Code

Broken code:

```
<html>
<body>
  <meta charset="UTF-8">
  <title>Bad Boilerplate</title>
</body>
<head>
  <h1>Hello</h1>
</head>
</html>
```

Problems:

1. `<!DOCTYPE html>` is missing.
2. The `html` element has no `lang` attribute.
3. `head` should come before `body`.
4. `meta charset` belongs in `head`, not `body`.
5. `title` belongs in `head`, not `body`.
6. `h1` belongs in `body`, not `head`.
7. The viewport meta tag is missing.

Fixed version:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Good Boilerplate</title>
  </head>

  <body>
    <h1>Hello</h1>
  </body>
</html>
```

Why this is correct:

- `DOCTYPE` is first.
- `html` wraps the whole document.
- `lang="en"` identifies the page language.

- `head` comes before `body`.
- Metadata is inside `head`.
- Visible content is inside `body`.
- The document has charset and viewport metadata.

---

36. Sprint 2 Practice Task

Create a file named:

```
index.html
```

Build a complete HTML page for:

```
Local Bakery
```

Requirements:

Inside the document, include:

- `<!DOCTYPE html>`.
- `<html lang="en">`.
- `head`.
- `body`.
- `meta charset="UTF-8"`.
- Viewport meta tag.
- `title`.
- Meta description.
- Stylesheet link.
- Favicon link.
- Open Graph title.
- Open Graph type.
- Open Graph image.
- Open Graph URL.
- One visible `h1`.
- One visible `p`.

Starter solution:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
```

```
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Local Bakery</title>
    <meta name="description" content="A simple landing page for a local
bakery.">

    <link rel="stylesheet" href="./styles.css">
    <link rel="icon" href="./favicon.ico">

    <meta property="og:title" content="Local Bakery">
    <meta property="og:type" content="website">
    <meta property="og:image" content="https://example.com/bakery-preview.png">
    <meta property="og:url" content="https://example.com">
  </head>

  <body>
    <h1>Local Bakery</h1>
    <p>We bake fresh bread, cakes, and pastries every morning.</p>
  </body>
</html>
```

---

37. Challenge Task

Add preconnect links for an external font service.

Example:

```
<link rel="preconnect" href="https://fonts.googleapis.com">
<link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
<link href="https://fonts.googleapis.com/css2?
family=Playwrite+CU:wght@100..400&display=swap" rel="stylesheet">
```

Important rule:

```
Do not put font styling in the HTML.
```

The HTML can link the font resource.

The CSS should decide how the font is used.

---

38. Sprint 2 Memory Template

First memorize this:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page Title</title>
  </head>

  <body>
    <h1>Page Heading</h1>
    <p>Page paragraph.</p>
  </body>
</html>
```

Then add this:

```
<meta name="description" content="Short, clear page description.">
<link rel="stylesheet" href="./styles.css">
<link rel="icon" href="./favicon.ico">
```

Then add this:

```
<meta property="og:title" content="Page Title">
<meta property="og:type" content="website">
<meta property="og:image" content="https://example.com/preview.png">
<meta property="og:url" content="https://example.com">
```

---

39. Sprint 2 Revision Questions

Use these questions to test yourself.

Question 1

What does `<!DOCTYPE html>` do?

Answer:

It tells the browser that the document uses modern HTML.

Question 2

Where should `<!DOCTYPE html>` go?

Answer:

At the very top of the HTML file, before the `html` element.

Question 3

What does the `html` element do?

Answer:

It wraps the whole HTML document.

Question 4

What does `lang="en"` mean?

Answer:

It means the main language of the page is English.

Question 5

What belongs inside the `head`?

Answer:

Metadata, title, charset, viewport, stylesheet links, favicon links, Open Graph tags, preconnect links, and script references.

Question 6

What belongs inside the `body`?

Answer:

Visible page content such as headings, paragraphs, images, links, forms, and tables.

Question 7

Why should visible content not go inside `head`?

Answer:

Because the `head` is for information about the page, not content shown to users.

Question 8

Why should UTF-8 be declared?

Answer:

It helps the browser display text, symbols, and many languages correctly.

Question 9

What does the viewport meta tag help with?

Answer:

It helps the page display properly on different screen sizes, especially mobile devices.

Question 10

What does the `title` element control?

Answer:

It controls the text shown in the browser tab.

Question 11

What does a meta description do?

Answer:

It gives a short description of the page, often useful for search result snippets.

Question 12

What does the `link` element do?

Answer:

It connects the HTML document to an external resource.

Question 13

What does `rel="stylesheet"` mean?

Answer:

It means the linked resource is a CSS stylesheet.

Question 14

What does `href` do on a `link` element?

Answer:

It gives the path or URL to the linked resource.

Question 15

What does a favicon link do?

Answer:

It connects the page to the small icon shown in the browser tab.

Question 16

What is preconnect used for?

Answer:

It tells the browser to prepare a connection to an external resource early.

Question 17

What are the four main Open Graph properties in Sprint 2?

Answer:

`og:title`, `og:type`, `og:image`, and `og:url`.

Question 18

What does the `script` element do?

Answer:

It can embed or link executable JavaScript code.

Question 19

Does Sprint 2 teach JavaScript?

Answer:

No. Sprint 2 only teaches how to recognize script references in HTML.

Question 20

What does separation of concerns mean?

Answer:

It means each technology has a separate job: HTML for structure, CSS for style, and JavaScript for behavior.

---

40. Sprint 2 Key Rules

Memorize these rules:

- `DOCTYPE` comes first.
- Use `<!DOCTYPE html>` for modern HTML.
- The `html` element wraps the page.
- Add `lang` to the `html` element.
- The `head` comes before the `body`.
- Metadata belongs in the `head`.
- Visible page content belongs in the `body`.
- Use `meta charset="UTF-8"` in every new HTML document.
- Use the viewport meta tag in modern pages.
- Use `title` for the browser tab.
- Use `h1` for the visible main heading.
- Use meta description for a short page summary.
- Use `link rel="stylesheet"` to connect CSS.
- Use `link rel="icon"` to connect a favicon.
- Use preconnect for preparing external resource connections.

- Use Open Graph tags for social preview metadata.
- Recognize `script src`, but do not study JavaScript yet.
- Keep HTML, CSS, and JavaScript responsibilities separate.

---

41. Sprint 2 Mini Glossary

Boilerplate

A reusable starting structure for a new HTML document.

`DOCTYPE`

A declaration that tells the browser to use modern HTML rules.

`html`

The root element that wraps the whole HTML page.

`lang`

An attribute that tells the main language of the page.

`head`

The part of the document that contains information about the page.

`body`

The part of the document that contains visible page content.

Metadata

Information about the page, usually placed in the `head`.

Charset

Character encoding information that helps the browser read text correctly.

UTF-8

A common character encoding that supports many languages and symbols.

Viewport

The visible area of a web page in the browser.

`title`

The element that sets the browser tab title.

Meta Description

A short description of the page.

`link`

A void element that connects external resources to the HTML document.

`rel`

An attribute that describes the relationship between the HTML page and the linked resource.

`href`

An attribute that gives the location of the linked resource.

Stylesheet

A CSS file that controls the appearance of the page.

Favicon

The small icon shown in the browser tab.

Preconnect

A hint that tells the browser to prepare a connection to an external resource early.

Open Graph

Metadata that helps control how a page appears when shared socially.

`script`

An element used to link or include JavaScript.

Separation of Concerns

The practice of keeping structure, style, and behavior separate.

---

42. Final Sprint 2 Example

This example combines the main Sprint 2 concepts:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">

    <title>Local Bakery</title>
    <meta name="description" content="A local bakery landing page with fresh
bread, cakes, and contact information.">

    <link rel="preconnect" href="https://fonts.googleapis.com">
    <link rel="preconnect" href="https://fonts.gstatic.com" crossorigin>
    <link href="https://fonts.googleapis.com/css2?
family=Playwrite+CU:wght@100..400&display=swap" rel="stylesheet">

    <link rel="stylesheet" href="./styles.css">
    <link rel="icon" href="./favicon.ico">

    <meta property="og:title" content="Local Bakery">
    <meta property="og:type" content="website">
    <meta property="og:image" content="https://example.com/bakery-preview.png">
    <meta property="og:url" content="https://example.com">

    <script src="./script.js"></script>
  </head>

  <body>
    <h1>Local Bakery</h1>
    <p>Fresh bread, cakes, and pastries made every morning.</p>
  </body>
</html>
```

This example includes:

- `DOCTYPE` first.
- `html lang="en"`.

- `head` before `body`.
- UTF-8 charset.
- Viewport meta tag.
- Browser tab title.
- Meta description.
- Preconnect links.
- External font stylesheet link.
- External CSS stylesheet link.
- Favicon link.
- Open Graph metadata.
- External script reference.
- Visible content inside `body`.

---

43. Sprint 2 Completion Standard

Sprint 2 is complete when you can write a full HTML document from memory.

You should be able to create this structure without looking:

```
<!DOCTYPE html>
<html lang="en">
  <head>
    <meta charset="UTF-8">
    <meta name="viewport" content="width=device-width, initial-scale=1.0">
    <title>Page Title</title>
    <meta name="description" content="Short, clear page description.">
    <link rel="stylesheet" href="./styles.css">
    <link rel="icon" href="./favicon.ico">
  </head>

  <body>
    <h1>Page Heading</h1>
    <p>Page paragraph.</p>
  </body>
</html>
```

You are ready to move on from Sprint 2 when you can:

- Explain what `DOCTYPE` does.
- Explain why `html lang` matters.
- Separate `head` and `body` correctly.
- Put metadata in `head`.
- Put visible content in `body`.
- Add `charset` and viewport meta tags.

- Add a useful page title.
- Add a clear meta description.
- Link a stylesheet.
- Link a favicon.
- Recognize preconnect links.
- Add basic Open Graph tags.
- Recognize external script references.
- Explain separation of concerns in simple words.
- Fix broken boilerplate code.

That is the main point of Sprint 2.